



Build Your Own Local RAG Engine

mercel-vercel.vercel.app

A Complete Step-by-Step Guide for Windows, macOS, and Linux Users

Table of Contents

1. Introduction: What This Guide Will Do
2. Part 1: Getting Started
3. Part 2: Setting Up Your System
4. Part 3: Running Your Application
5. Part 4: Troubleshooting
6. Part 5: Making It Yours
7. Part 6: Advanced Configuration
8. Part 7: Everyday Use
9. Conclusion

If you find all of this overwhelming, or dont have time, you can hire us to do it, and we can set all of this up for your business. Feel free to contact us including details of your current setup, and what your goals are, and we will be in contact with you with a quote.

****Contact us at: mercel-vercel.vercel.app to learn more.**

Introduction: What This Guide Will Do

This guide will walk you through building a **private AI knowledge base** that runs entirely on your own computer. No cloud services. No subscriptions. No technical expertise required.

By the end of this guide, you will have a **sleek web application** that: - Answers questions based on your documents - Runs 24/7 on your hardware - Keeps all your data private - Costs \$0 per month

Who is this guide for? - Business professionals - Small business owners - Consultants and freelancers - Anyone who wants a private AI assistant

What you need: - A computer (Windows, macOS, or Linux) - 16 GB RAM recommended (8 GB minimum) - About 10 GB of free disk space - Basic ability to follow instructions

What you'll build: - A private AI system that reads your documents and answers questions - A beautiful web interface that looks professional - A system that runs completely on your own hardware

Part 1: Getting Started

Step 1: Install Python

What is Python?

Python is a programming language that runs many applications. You'll need it to run your AI system.

For Windows Users:

1. Go to python.org/downloads
2. Click the "Download Python" button (the latest version).
3. Run the installer.
4. **IMPORTANT:** Check the box that says "**Add Python to PATH**" at the bottom of the first screen.
5. Click "Install Now."
6. Wait for the installation to complete.
7. Click "Close."

For macOS Users:

1. Go to python.org/downloads
2. Download the macOS installer.
3. Open the .pkg file.
4. Follow the installation prompts.
5. Click “Install” and enter your password when prompted.

For Linux Users (Ubuntu/Debian):

Open a terminal and type:

```
sudo apt update
sudo apt install python3 python3-pip -y
```

How to verify Python is installed:

Open a terminal and type:

Operating System	Command
Windows	python --version
macOS	python3 --version
Linux	python3 --version

You should see something like Python 3.11.x.

If you get an error on Windows:

The most common error is:

'python' is not recognized as an internal or external command

Fix: Reinstall Python and check “Add to PATH.” Or use py instead of python:

```
py --version
```

Step 2: Install Ollama

What is Ollama?

Ollama is a program that runs AI models on your computer. It's the “engine” that powers your AI.

For Windows Users:

1. Go to ollama.com/download

2. Click “Download for Windows.”
3. Run the `OllamaSetup.exe` file.
4. Follow the installation prompts.
5. Ollama starts automatically in the background.

For macOS Users:

1. Go to ollama.com/download
2. Click “Download for macOS.”
3. Open the downloaded .zip file.
4. Drag Ollama to your Applications folder.
5. Open Ollama from your Applications folder.

For Linux Users (Ubuntu/Debian):

Open a terminal and type:

```
curl -fsSL https://ollama.com/install.sh | sh
```

How to verify Ollama is installed:

Open a terminal and type:

Operating System	Command
Windows	<code>ollama --version</code>
macOS	<code>ollama --version</code>
Linux	<code>ollama --version</code>

You should see a version number.

Step 3: Pull Your AI Model

What is a model?

A model is the “brain” that powers your AI. We’ll use a model called “Qwen 2.5” because it’s fast and good for business documents.

Open a terminal and type:

Operating System	Command
Windows	<code>ollama pull qwen2.5:7b</code>
macOS	<code>ollama pull qwen2.5:7b</code>
Linux	<code>ollama pull qwen2.5:7b</code>

This will download about 5 GB of data. Wait for it to finish.

How to verify your model is installed:

Type:

```
ollama list
```

You should see qwen2.5:7b in the list.

Step 4: Create Your Project Folder

What is a project folder?

This is where all your files will live. It keeps everything organized.

For Windows Users:

Open PowerShell and type:

```
mkdir C:\Users\YourName\RAG-Engine  
cd C:\Users\YourName\RAG-Engine
```

Replace YourName with your actual Windows username.

For macOS Users:

Open Terminal and type:

```
mkdir -p ~/RAG-Engine  
cd ~/RAG-Engine
```

For Linux Users:

Open Terminal and type:

```
mkdir -p ~/RAG-Engine  
cd ~/RAG-Engine
```

Step 5: Create Your Knowledge Base Folder

What is the knowledge base?

This is where you put the documents your AI will read.

For Windows Users:

`mkdir knowledgebase_database`

For macOS/Linux Users:

`mkdir knowledgebase_database`

What goes in this folder?

Any text-based documents you want your AI to understand: - PDF files - Word documents (.docx) - Markdown files (.md) - Text files (.txt)

Example documents to add: - Your business policies - Product catalogs - Customer FAQs - Meeting notes - Industry reports

How to add documents:

Simply copy and paste your files into this folder using your file manager (Finder on Mac, File Explorer on Windows).

Part 2: Setting Up Your System

Step 6: Install Required Software Packages

What are packages?

Think of them as “add-ons” that give your system specific abilities.

Open a terminal and navigate to your project folder:

Operating System	Command
Windows	<code>cd C:\Users\YourName\RAG-Engine</code>
macOS/Linux	<code>cd ~/RAG-Engine</code>

Now install the required packages:

Operating System	Command
Windows	<code>pip install streamlit llama-index llama-index-llms-ollama llama-index-embeddings-huggingface huggingface-hub sentence-transformers</code>
macOS	<code>pip3 install streamlit llama-index llama-index-llms-ollama</code>

Operating System	Command
Linux	<pre>llama-index-embeddings-huggingface huggingface-hub sentence-transformers</pre>
	<pre>pip3 install streamlit llama-index llama-index-llms-ollama llama-index-embeddings-huggingface huggingface-hub sentence-transformers</pre>

Wait for the installation to finish. This might take a few minutes.

Common installation errors and fixes:

Error	Fix
“pip not found”	Reinstall Python and check “Add to PATH” (Windows)
“Permission denied”	Add <code>sudo</code> before the command (macOS/Linux)
“Connection timeout”	Try again after a few minutes

Step 7: Create Your Application File

What is this file?

This is the heart of your system. It’s the web interface that will appear in your browser.

Where to create it:

Create a file at this location:

Operating System	File Location
Windows	C:\Users\YourName\RAG-Engine\app.py
macOS/Linux	~/RAG-Engine/app.py

How to create the file:

1. Open your text editor (Notepad on Windows, TextEdit on Mac, or any code editor like VS Code).
2. Copy and paste the code below.

3. Save the file as app.py in your RAG-Engine folder.

The code:

```
# ~/RAG-Engine/app.py
# This is the main application file for your AI Knowledge System

import streamlit as st
import os
from llama_index.core import VectorStoreIndex,
    SimpleDirectoryReader, Settings
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# Page Configuration
st.set_page_config(
    page_title="Business Analyst",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for a Sleek Professional Interface
st.markdown("""
<style>
    .main-header {
        text-align: center;
        padding: 2rem 0;
        background: linear-gradient(135deg, #667eea 0%, #764ba2
            100%);
        border-radius: 10px;
        margin-bottom: 2rem;
        color: white;
    }
    .main-header h1 {
        font-size: 2.5rem;
        margin-bottom: 0.5rem;
    }
    .main-header p {
        font-size: 1.1rem;
        opacity: 0.9;
    }
    .stChatMessage {
        border-radius: 10px;
        padding: 1rem;
        margin-bottom: 1rem;
        box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }
    .model-info {
        background: #f0f2f6;
        border-radius: 8px;

```



```

        padding: 1rem;
        margin-bottom: 1rem;
        font-size: 0.9rem;
    }
</style>
""" , unsafe_allow_html=True)

# Title Section
st.markdown("""
<div class="main-header">
    <h1>👋 Business Analyst</h1>
    <p>Your Private AI Knowledge Assistant</p>
</div>
""" , unsafe_allow_html=True)

# Sidebar Configuration
with st.sidebar:
    st.header("System Status")

    st.markdown('<div class="model-info">',
        unsafe_allow_html=True)
    st.markdown("***Model:** Qwen 2.5 7B")
    st.markdown("***Embeddings:** BAAI/bge-small-en-v1.5")
    st.markdown("***Runtime:** Ollama (Port 11434)")
    st.markdown('</div>', unsafe_allow_html=True)

    st.divider()

    # Knowledge Base Status
    db_path = "./knowledgebase_database"
    if os.path.exists(db_path) and os.listdir(db_path):
        doc_count = len(os.listdir(db_path))
        st.success(f"🟢 Knowledge Base Ready ({doc_count} files)")
    else:
        st.warning("⚠ Knowledge Base Empty")
        st.info("Place files in the 'knowledgebase_database' folder")

    st.divider()

    # How to Use
    st.markdown("### How to Use")
    st.markdown("""
1. Place documents in `knowledgebase_database`
2. The system automatically indexes them
3. Ask questions in the chat below
4. The AI uses your documents to answer
""")

    st.divider()

    # Clear Chat Button

```

```

    if st.button("Clear Chat History", use_container_width=True):
        st.session_state.messages = []
        st.rerun()

# Configure Local Models
@st.cache_resource
def load_models():
    Settings.llm = Ollama(
        model="qwen2.5:7b",
        request_timeout=120.0,
        base_url="http://localhost:11434"
    )
    Settings.embed_model = HuggingFaceEmbedding(
        model_name="BAAI/bge-small-en-v1.5"
    )
    return Settings

# Load Models
settings = load_models()

# The Master System Prompt
SYSTEM_PROMPT = """
### ROLE & CORE PHILOSOPHY
You are a Business Analyst with deep expertise in business
operations, process optimization, and strategic planning.
Your purpose is to analyze business documents, extract
insights, and provide actionable recommendations. You are
precise, professional, and thorough in your analysis.

### THE TRANSLATION MATRIX
When analyzing any document or answering questions, follow this
framework:
1. Context: Identify the business scenario and relevant
background.
2. Key Points: Extract the most important information and
data.
3. Insights: Provide meaningful analysis and interpretation.
4. Recommendations: Suggest actionable next steps.

### INSTRUCTIONS
- Always base your responses on the documents provided in the
knowledge base.
- If the information is not in the documents, state clearly that
you don't have that information.
- Use clear, professional language suitable for business
communication.
- When providing recommendations, consider feasibility and
potential impact.
- If the user asks a question outside the knowledge base,
acknowledge the limitation and suggest adding relevant
documents.

### RESPONSE FORMAT
- Use bullet points for clarity when listing items.

```

- Use headings to organize longer responses.
- Be concise but comprehensive.

```

"""

# Build Index
@st.cache_resource
def build_index():
    if not os.path.exists(db_path) or not os.listdir(db_path):
        return None

    documents = SimpleDirectoryReader(db_path).load_data()
    index = VectorStoreIndex.from_documents(documents)
    return index

index = build_index()

# Chat Interface
if index:
    query_engine =
        index.as_query_engine(system_prompt=SYSTEM_PROMPT)

    # Initialize Chat History
    if "messages" not in st.session_state:
        st.session_state.messages = []

    # Display Chat History
    for message in st.session_state.messages:
        with st.chat_message(message["role"]):
            st.markdown(message["content"])

    # Chat Input
    if prompt := st.chat_input("Ask a question regarding your
        business..."):
        # Add user message
        st.session_state.messages.append({"role": "user",
            "content": prompt})
        with st.chat_message("user"):
            st.markdown(prompt)

        # Generate response
        with st.chat_message("assistant"):
            message_placeholder = st.empty()
            with st.spinner("Analyzing..."):
                response = query_engine.query(prompt)
                message_placeholder.markdown(response.response)

        # Add assistant message
        st.session_state.messages.append({"role": "assistant",
            "content": response.response})
else:
    # Empty State
    st.info("""

```

📖 Getting Started

1. Create a folder called `knowledgebase\_database` in your project directory.
2. Place your business documents (PDF, Markdown, Text) inside.
3. Refresh this page to load the documents.

The system will automatically index your files and make them searchable.

```
"""
```

```
if st.button("Refresh Page"):  
    st.rerun()
```

Part 3: Running Your Application

Step 8: Start Your Application

Before you start:

1. Make sure Ollama is running.
2. Make sure you have documents in your knowledgebase_database folder.

How to start your app:

Open a terminal and navigate to your project folder:

Operating System	Command
Windows	cd C:\Users\YourName\RAG-Engine
macOS/Linux	cd ~/RAG-Engine

Start the application:

Operating System	Command
Windows	streamlit run app.py
macOS	streamlit run app.py
Linux	streamlit run app.py

This will start a local server. Your web browser will open automatically showing your AI assistant.

If the browser doesn't open automatically:

Go to your web browser and type:

`http://localhost:8501`

Step 9: Test Your System

Now that your app is running, you can test it.

- 1. Ask a question:** In the chat input at the bottom, type a question about your business documents.
 - Example: “What is our refund policy?”
 - Example: “What services do we offer?”
 - 2. Wait for the response:** The AI will analyze your documents and provide an answer.
 - 3. If you get an answer:** Great! Your system is working.
 - 4. If you get an error:** See the troubleshooting section below.
-

Part 4: Troubleshooting

Common Problems and Solutions

Problem 1: “Module not found” errors

Symptom: You see an error like

`ModuleNotFoundError: No module named 'streamlit'`

Solution: You need to install the packages. In your terminal:

Operating System	Command
Windows	<code>pip install streamlit llama-index llama-index-llms-ollama llama-index-embeddings-huggingface huggingface-hub sentence-transformers</code>
macOS	<code>pip3 install streamlit llama-index llama-index-llms-ollama</code>

Operating System	Command
Linux	llama-index-embeddings-huggingface huggingface-hub sentence-transformers
	pip3 install streamlit llama-index llama-index-llms-ollama
	llama-index-embeddings-huggingface huggingface-hub sentence-transformers

Problem 2: “Ollama not found” or “Connection refused”

Symptom: You see an error like Connection refused OR Cannot connect to Ollama

Solution: Make sure Ollama is running. Open a terminal and type:

```
ollama serve
```

Keep this terminal open. In another terminal, run your app again.

Problem 3: “No documents found”

Symptom: The app shows “Knowledge Base Empty” or “No documents found”

Solution: Make sure your documents are in the correct folder:

Operating System	Folder Location
Windows	C:\Users\YourName\RAG-Engine\knowledgebase_database
macOS/Linux	~/RAG-Engine/knowledgebase_database

Problem 4: “Model not found”

Symptom: You see an error about the model not being available

Solution: Pull the model again:

```
ollama pull qwen2.5:7b
```

Problem 5: App won't start

Symptom: `streamlit run app.py` gives an error

Solution: Check that you're in the correct folder:

Operating System	Command
Windows	<code>cd C:\Users\YourName\RAG-Engine</code>
macOS/Linux	<code>cd ~/RAG-Engine</code>

Problem 6: Windows-specific “Python not recognized”

Symptom: 'python' is not recognized as an internal or external command

Solution: Use `py` instead of `python`:

```
py --version
```

Or reinstall Python and check “Add to PATH.”

Problem 7: macOS “pip not found”

Symptom: `pip: command not found`

Solution: Use `pip3` instead of `pip`:

```
pip3 install streamlit llama-index
```

Problem 8: macOS “externally managed environment” error

Symptom: You see an error like `externally-managed-environment`

Solution: Create a virtual environment or use `--break-system-packages`:

```
pip3 install --break-system-packages streamlit llama-index
```

Part 5: Making It Yours

Customizing Your System

Changing the AI Model

If you want to use a different model, edit the code in `app.py`:

Find this line:

```
Settings.llm = Ollama(model="qwen2.5:7b", ...)
```

Change `qwen2.5:7b` to another model you've pulled. For example:

```
Settings.llm = Ollama(model="llama3.1:8b", ...)
```

To pull a new model:

```
ollama pull llama3.1:8b
```

Changing the System Prompt

The system prompt tells the AI how to behave. Find the `SYSTEM_PROMPT` section in `app.py` and modify it to match your needs.

Example if you want a customer service assistant:

```
You are a customer service representative. Always be polite and helpful.
```

Changing the Colors and Design

The visual design is controlled by the CSS section. You can change colors, fonts, and layout to match your brand.

Example - Change the header background color:

Find this line in the CSS:

```
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

Change the colors to match your brand. For example:

```
background: linear-gradient(135deg, #1e3a8a 0%, #3b82f6 100%);
```

Part 6: Advanced Configuration

Configuring the Vector Database

By default, LlamaIndex stores vectors in memory. For a more permanent solution, you can configure it to use SQLite or Qdrant.

Using SQLite (Simple)

Add this to your `app.py`:

```
# ~/RAG-Engine/app.py (add these lines)
import sqlite_vec
from llama_index.vector_stores.sqlitevec import SQLiteVectorStore

# Create a SQLite vector store
vector_store = SQLiteVectorStore(database_path="./vector_store.db")
index = VectorStoreIndex.from_documents(documents,
                                       storage_context=vector_store)
```

To install SQLite support:

```
pip install sqlite-vec
```

Using Qdrant (Scalable)

If you have a large number of documents, use Qdrant:

```
# ~/RAG-Engine/app.py (add these lines)
from llama_index.vector_stores.qdrant import QdrantVectorStore
from qdrant_client import QdrantClient

# Create a Qdrant client
client = QdrantClient(path="./qdrant_data")
vector_store = QdrantVectorStore(client=client,
                                collection_name="business_docs")
index = VectorStoreIndex.from_documents(documents,
                                       storage_context=vector_store)
```

To install Qdrant support:

```
pip install qdrant-client
```

Optimizing Performance

Adjusting Context Window

The context window determines how much information the AI considers. Add this to your app.py:

```
Settings.context_window = 4096
```

Adjusting Chunk Size

In your app.py, add:

```
from llama_index.core.node_parser import SimpleNodeParser

parser = SimpleNodeParser.from_defaults(chunk_size=512,
                                       chunk_overlap=50)
nodes = parser.get_nodes_from_documents(documents)
index = VectorStoreIndex(nodes)
```

Part 7: Everyday Use

Adding New Documents

1. Copy your new documents into the knowledgebase_database folder.
 2. Refresh your browser.
 3. The system will automatically index the new documents.
-

Backing Up Your Data

Your knowledge base is valuable. Back it up regularly.

What to back up:

Item	Location
Knowledge Base	knowledgebase_database folder
Application File	app.py
Vector Database	vector_store.db (if using SQLite)

How to back up:

Copy these files to an external drive or cloud storage.

Operating System	Backup Command
Windows	Copy-Item -Path C:\Users\YourName\RAG-Engine -Destination D:\Backup -Recurse
macOS/Linux	cp -r ~/RAG-Engine /Volumes/ Backup/

Conclusion

You now have a private, AI-powered knowledge system that: - Runs on your own hardware - Keeps all your data private - Costs \$0 per month - Answers questions based on your documents - Is customizable to your needs

What you've learned: - How to install Python and Ollama - How to pull an AI model - How to create a knowledge base folder - How to create a web application - How to run and test your system - How to troubleshoot common issues

Remember: This is your system. Take time to explore it, add documents, and customize the interface to match your business. The more you use it, the more valuable it becomes.

If you find all of this overwhelming, or dont have time, you can hire us to do it, and we can set all of this up for your business. Feel free to contact us including details of your current setup, and what your goals are, and we will be in contact with you with a quote.

****Contact us at: mercel-vercel.vercel.app to learn more.**

About Mercel

Mercel is dedicated to helping businesses leverage AI technology for sustainable growth. We build custom AI systems that run on your own hardware, ensuring your data never leaves your servers.

Visit us at mercel-vercel.vercel.app to learn more.

This guide was created using a local AI assistant running on a personal server.